

14 .Mini Project: Cybersecurity Solution Development: Design and implement a basic security solution such as intrusion detection or secure communication system.

Mini Project Guideline

Cybersecurity Solution Development

Since Practicals **1–13** have already covered security tools, classical encryption, AES, RSA, hashing, Nmap, Wireshark, firewalls, attack simulation, IDS, vulnerability assessment, SSL/TLS, and digital signatures, Practical 14 should work as an **integrated mini-project** that combines the concepts students have learned.

The project can be completed using **Google Colab**, along with tools/platforms such as **I4C**, **IP India**, Nmap/Wireshark where appropriate. The project should be **defensive and educational**, using only authorized or simulated data.

1. Mini Project Title

Cybersecurity Solution Development

Design and implement a basic cybersecurity solution such as an Intrusion Detection System or Secure Communication System.

2. Purpose of the Mini Project

The purpose of this mini-project is to give students practical experience in applying cybersecurity concepts learned during Practicals 1–13 to a small, working security solution.

Students should:

- Identify a cybersecurity problem.
- Design a basic solution.
- Implement the solution.

- Test it using sample/simulated data.
- Analyze the results.
- Document the security controls used.
- Present the project through a report and demonstration.

3. Important Guideline for Students

Students should NOT simply repeat one previous practical.

For example:

✗ Only implementing AES again is not a mini-project.

✗ Only running Nmap and showing the output is not a complete mini-project.

Instead, students should **combine multiple concepts**.

Example

A student can develop:

Basic Network Intrusion Detection System using Python

and use:

- Python
- Google Colab
- CSV/network traffic dataset
- Hashing
- IP/port analysis
- Rule-based detection
- Visualization
- Alert generation
- I4C resources for cybersecurity awareness/context

This makes the project an extension of Practicals 6, 7, 10 and 11.

4. Recommended Mini Project Topics

Students can select **ONE** topic.

No.	Mini Project Topic	Main Practicals Used
1	Basic Network Intrusion Detection System	6, 7, 10, 11
2	Secure File Transfer System	3, 4, 12, 13
3	Password Security & Strength Analyzer	5, 11
4	Phishing URL Detection System	5, 10, 11
5	Network Port Monitoring System	6, 8, 10
6	Secure Communication Demonstrator	3, 4, 12, 13
7	File Integrity Monitoring System	5, 13
8	Basic Security Log Analyzer	7, 10, 11
9	Suspicious IP Detection System	6, 7, 10
10	Cybersecurity Awareness & Incident Reporting System	10, 11 + I4C

5. Recommended Project for Google Colab

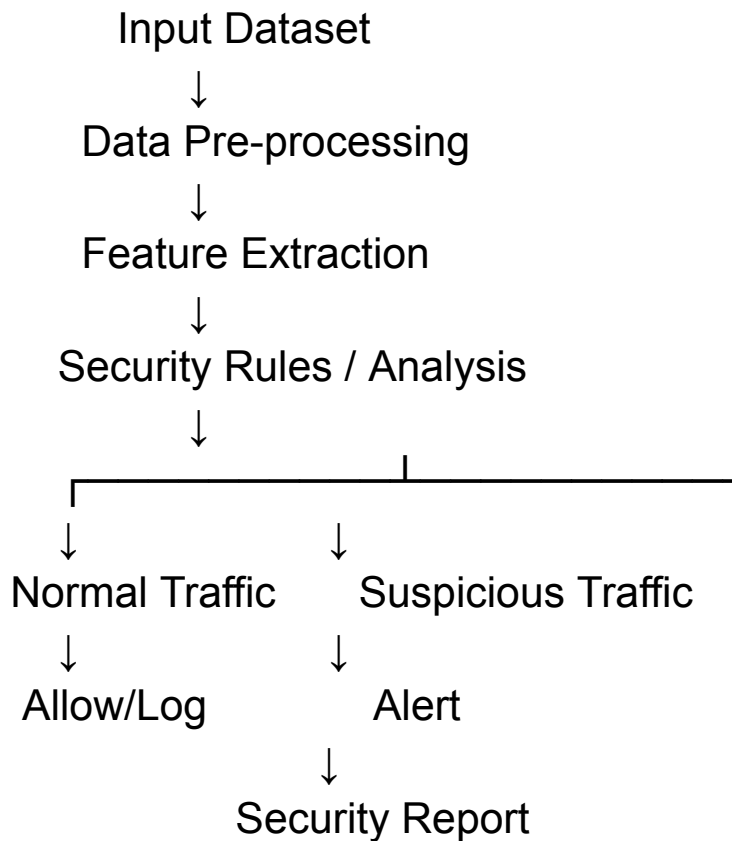
For students working mainly in **Google Colab**, the easiest project to implement and demonstrate is:

Basic Network Intrusion Detection System Using Python

Project objective

To develop a basic rule-based intrusion detection system that analyzes sample network-traffic data and identifies potentially suspicious activities.

6. Project Architecture



7. Tools and Technologies

Tool/Technology	Purpose
Google Colab	Development and execution
Python	Programming
Pandas	Dataset processing
NumPy	Data processing
Matplotlib	Visualization

CSV Dataset	Network traffic input
Hashlib	Hash/integrity functions
Nmap	Authorized network scanning
Wireshark	Packet analysis
I4C resources	Cybercrime awareness/reporting context
IP India	Intellectual-property/copyright/trademark awareness where relevant

8. How Previous Practicals Connect to Mini Project

Students have already completed:

Practical 1

Security Tools & Environment

Students learned how to prepare their cybersecurity environment.

Practical 2

Classical Encryption

Students learned basic encryption concepts.

Practical 3

AES

Students learned symmetric encryption.

Practical 4

RSA

Students learned asymmetric encryption.

Practical 5

Hashing & Password Security

Students learned hashing and password verification.

Practical 6

Nmap

Students learned authorized network/port discovery.

Practical 7

Wireshark

Students learned packet and network-traffic analysis.

Practical 8

Firewall

Students learned traffic-control concepts.

Practical 9

Attack Simulation

Students learned how security attacks can be represented in a controlled environment.

Practical 10

IDS

Students learned intrusion detection concepts.

Practical 11

Vulnerability Assessment

Students learned basic vulnerability identification.

Practical 12

SSL/TLS

Students learned secure communication.

Practical 13

Digital Signature

Students learned authentication and integrity verification.

Practical 14

Now students should **combine these concepts into one small cybersecurity solution.**

9. Mini Project Development Stages

Stage 1 – Problem Identification

Students should first identify a simple cybersecurity problem.

Example:

Network administrators need a simple mechanism to identify potentially suspicious network traffic from a dataset.

Stage 2 – Define Objectives

Example objectives:

1. Read network traffic data.
2. Process the dataset.
3. Identify suspicious traffic based on predefined rules.
4. Generate security alerts.
5. Display statistics and graphs.
6. Produce a final security report.

10. Stage 3 – Dataset

Students can use a **public cybersecurity dataset** or instructor-provided simulated data.

For Google Colab, a CSV file is convenient.

Example columns:

Source_IP
Destination_IP
Source_Port
Destination_Port
Protocol
Packet_Count
Connection_Duration
Status

Example:

Source IP	Destination Port	Protocol	Packets	Status
192.168.1.10	80	TCP	25	Normal
192.168.1.20	443	TCP	30	Normal
192.168.1.50	22	TCP	500	Suspicious
192.168.1.60	23	TCP	700	Suspicious

Students should clearly mention the **source and license/usage terms of any external dataset** in their report.

11. Stage 4 – Load Dataset in Google Colab

```
import pandas as pd
```

```
df = pd.read_csv("network_traffic.csv")
```

```
print("Dataset loaded successfully")  
print(df.head())
```

12. Stage 5 – Analyze Dataset

```
print("Dataset Shape:")  
print(df.shape)
```

```
print("\nColumn Names:")  
print(df.columns)
```

```
print("\nMissing Values:")  
print(df.isnull().sum())
```

13. Stage 6 – Basic Security Rules

For an educational IDS, students can create simple rules.

Example:

```
def detect_suspicious(row):
```

```
    if row["Packet_Count"] > 500:  
        return "Suspicious"
```

```
    if row["Destination_Port"] in [23, 445]:  
        return "Suspicious"
```

```
    return "Normal"
```

```
df["Detection"] = df.apply(detect_suspicious, axis=1)
```

```
print(df[["Source_IP", "Destination_Port", "Detection"]])
```

Important

These are **illustrative rules**, not a real-world IDS detection engine. Students should explain that thresholds must be validated against an appropriate dataset and environment.

14. Stage 7 – Generate Alerts

```
alerts = df[df["Detection"] == "Suspicious"]

print("Number of Suspicious Records:", len(alerts))

print("\nSecurity Alerts:")
print(alerts)
```

15. Stage 8 – Visualization

```
import matplotlib.pyplot as plt

df["Detection"].value_counts().plot(kind="bar")

plt.title("Network Traffic Security Analysis")
plt.xlabel("Traffic Category")
plt.ylabel("Number of Records")
plt.show()
```

This gives students a graphical representation of:

- Normal traffic
- Suspicious traffic

16. Stage 9 – Generate Security Report

```
total_records = len(df)
suspicious_records = len(
    df[df["Detection"] == "Suspicious"]
)
```

```
print("CYBERSECURITY ANALYSIS REPORT")
print("-----")
print("Total Records:", total_records)
print("Suspicious Records:", suspicious_records)
print("Normal Records:", total_records - suspicious_records)
```

17. Stage 10 – Save Results

```
df.to_csv(
    "intrusion_detection_results.csv",
    index=False
)
```

```
print("Security analysis saved successfully.")
```

Students can then show the generated result file in the project demonstration.

18. Optional Integration with Previous Practicals

Students who want a more advanced project can integrate additional security concepts.

Example:

Network Traffic



IDS Detection



Suspicious Event



Hash Event Information



Digital Signature



Secure Alert Record

This combines:

- IDS
- Hashing
- Digital signatures
- Secure communication

from Practicals **5, 10, 12 and 13.**

19. Alternative Mini Project – Secure Communication System

Students who prefer cryptography can select:

Secure Communication System Using AES + RSA + SHA-256 + Digital Signature

Working

Sender



Original Message



SHA-256 Hash



Digital Signature



AES Encryption



RSA-based Key Protection



Secure Transmission



Receiver



AES Decryption



Signature Verification



Integrity Verification

This project directly combines Practicals:

3 + 4 + 5 + 12 + 13

20. I4C Usage Guideline

Students may refer to the **Indian Cyber Crime Coordination Centre (I4C)** for the project's cybersecurity-awareness and incident-reporting context.

For example, students can include a section:

"Cyber Incident Awareness"

- Types of cybercrime
- Importance of reporting cyber incidents
- Basic precautions
- Evidence preservation
- Responsible cybersecurity practices

Students should use current official I4C information rather than copying unverified information from social media or random websites.

21. IP India Usage Guideline

IP India can be referenced when the project includes intellectual-property considerations.

Students can discuss:

- Copyright of software/code
- Trademark considerations
- Ownership of project material
- Proper attribution of datasets/images
- Open-source software licenses

Important

Students should **not copy code, datasets, logos, images, or documents without checking the applicable license/permission.**

22. Cybersecurity Ethics Guidelines

Students must follow these rules:

Allowed

- ✓ Use your own computer/network.
- ✓ Use instructor-provided lab environments.
- ✓ Use simulated attack data.
- ✓ Use public datasets according to their licenses.
- ✓ Analyze previously captured/sample traffic.
- ✓ Perform authorized security testing.

Not allowed

- ✗ Scan systems without authorization.
- ✗ Attack real websites.
- ✗ Attempt to obtain other people's passwords.
- ✗ Collect personal information without permission.
- ✗ Perform unauthorized DoS/DDoS activity.
- ✗ Attempt to bypass security controls on real systems.

The mini-project should demonstrate **defensive cybersecurity**, not unauthorized attacks.

23. Mini Project Report Format

Students should prepare approximately **15–25 pages**, depending on the institution's requirements.

Suggested structure

Chapter	Content
1	Cover Page
2	Certificate
3	Acknowledgement
4	Abstract
5	Introduction
6	Problem Statement
7	Objectives
8	Existing System
9	Proposed System
10	Tools & Technologies
11	System Architecture
12	Methodology
13	Implementation
14	Source Code
15	Screenshots
16	Testing
17	Results
18	Security Analysis
19	Limitations
20	Future Scope
21	Conclusion
22	References

24. Screenshots Students Should Include

For the **IDS project**, students should capture:

1. Google Colab notebook.
2. Dataset loaded successfully.
3. Dataset preview.
4. Data analysis output.
5. Detection rules/code.
6. Suspicious traffic output.
7. Security alerts.
8. Graph/chart.
9. Final analysis report.
10. Generated CSV/result file.

25. Testing Table

Test Case	Input	Expected Result	Actual Result
TC01	Normal traffic	Normal	Normal
TC02	High packet count	Suspicious	Suspicious
TC03	Normal port	Normal	Normal
TC04	Suspicious port	Suspicious	Suspicious
TC05	Empty/invalid data	Handle error	Tested
TC06	Multiple records	Generate report	Tested

Students should replace the **Actual Result** with what they actually observe.

26. Limitations

Students should mention that a basic student project has limitations, such as:

- Rule-based detection can produce false positives.
- A small dataset may not represent real-world traffic.
- It may not detect unknown attack patterns.
- It is not a replacement for a professional IDS.
- Results depend on the quality of the dataset and detection rules.

27. Future Scope

The project can be extended by adding:

- Machine-learning-based detection
- Real-time log monitoring
- Email/security alerts
- Dashboard visualization
- More traffic features
- Threat-intelligence integration
- Automated reporting
- Role-based access control
- Secure communication between components

28. Expected Learning Outcomes

After completing the mini-project, students should be able to:

1. Identify a basic cybersecurity problem.
2. Select appropriate security techniques.
3. Develop a small security application.
4. Analyze cybersecurity datasets.
5. Implement basic detection mechanisms.
6. Understand authentication and integrity.
7. Use cybersecurity tools responsibly.
8. Document and present security findings.
9. Understand the limitations of a basic security solution.
10. Apply concepts from Practicals 1–13 in an integrated project.

29. Mini Project Evaluation Guideline

A simple **100-mark evaluation** can be used:

Component	Marks
Problem identification	10
Objectives & design	10
Technical implementation	25
Integration of cybersecurity concepts	15
Testing & results	10
Report/documentation	15
Presentation/demo	10
Viva	5
Total	100

30. Student Submission Checklist

Before submission, every student should verify:

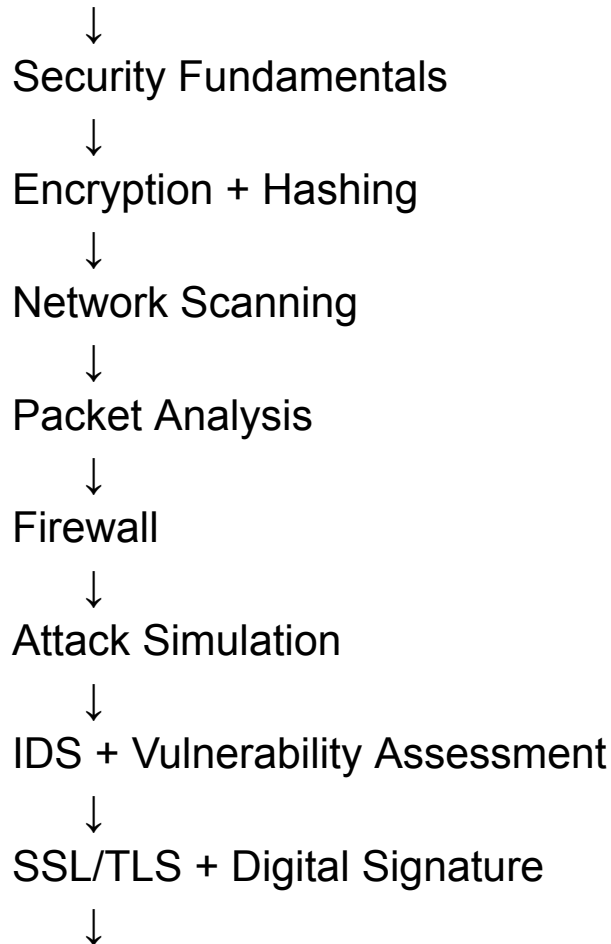
- Project title finalized
- Problem statement written
- Objectives defined
- Tools listed
- Architecture diagram prepared
- Methodology explained
- Code implemented in Google Colab
- Dataset/source documented
- Screenshots added
- Testing completed
- Results documented
- Limitations mentioned
- Future scope mentioned
- Conclusion written

- References added
- Viva preparation completed

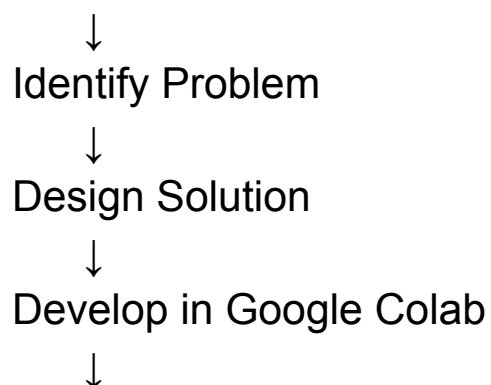
Final Recommended Project Flow

For your particular practical list, I would structure **Practical No. 14** like this:

PRACTICALS 1–13



MINI PROJECT



Test with Authorized/Simulated Data



Analyze Results



Security Report



Presentation + Viva

Recommended student project

“Basic Network Intrusion Detection and Security Analysis System Using Python”

This is particularly suitable because it naturally connects **Nmap + Wireshark + IDS + vulnerability assessment + hashing/digital-signature concepts** and can be implemented and demonstrated without requiring students to attack a real system.